

Design and Implementation of a UART Transmitter: A Complete RTL-to-GDSII ASIC Flow

Author: Manideep Emmadi

College: Methodist College of Engineering and Technology

Date: April 2026

Technology: SkyWater 130nm Open-Source PDK

Environment: Linux Mint / Docker

Abstract

This project demonstrates a complete "RTL-to-GDSII" implementation of a UART Transmitter using the SkyWater 130nm Open-Source Process Design Kit (PDK). The primary objective was to execute a top-to-bottom ASIC design flow, transforming a high-level Verilog description into a manufacture-ready silicon layout.

The methodology began with logic design and functional verification using Xilinx Vivado. Following successful logic testing, the design was synthesized with Yosys and moved into the OpenLane physical design environment. The automated flow managed critical stages including floorplanning, placement, Clock Tree Synthesis (CTS), and routing. To ensure manufacture readiness, physical verification was performed using Magic VLSI for Design Rule Checks (DRC) and Netgen for Layout vs. Schematic (LVS) sign-off. The project concluded with a post-layout simulation using Icarus Verilog and GTKWave, confirming that the design remains functionally correct even after accounting for the parasitic delays of the physical interconnects. This implementation successfully proves the reliability of an open-source EDA toolchain for producing verified silicon blueprints.

Keywords: UART Transmitter, Sky130, OpenLane, ASIC Flow, GDSII, Post-Layout Simulation, Physical Verification.

1. Introduction

The rapid evolution of Integrated Circuit (IC) technology has transformed the landscape of modern electronics, shifting the focus from discrete components to highly integrated System-on-Chip (SoC) architectures. Central to this evolution is the Application-Specific Integrated Circuit (ASIC) design flow, a rigorous process that translates abstract hardware descriptions into physical silicon. This project documents the complete RTL-to-GDSII implementation of a Universal Asynchronous Receiver-Transmitter (UART) module, utilizing the open-source SkyWater 130nm Process Design Kit (PDK) and the OpenLane automated flow.

1.1. Project Motivation

In the field of Very Large-Scale Integration (VLSI), the transition from "Front-End" logic design to "Back-End" physical implementation is one of the most challenging phases of hardware engineering. While Field Programmable Gate Arrays (FPGAs) offer a flexible platform for prototyping, ASIC design provides a deeper understanding of the physical constraints of silicon, including area optimization, power consumption, and timing integrity. This project was motivated by the desire to bridge the gap between high-level Verilog coding and physical silicon blueprints, ensuring that the theoretical logic is manufacture-ready and physically verifiable.

1.2. Role of the UART Protocol

The design vehicle chosen for this implementation is a UART Transmitter. Despite the emergence of high-speed protocols like USB and PCIe, UART remains a fundamental building block in embedded systems due to its simplicity and reliability in asynchronous communication. It serves as an ideal candidate for this study because it requires precise timing control and a robust Finite State Machine (FSM) to manage data serialization. Implementing a UART module through an ASIC flow provides excellent insights into how clock distribution and routing parasitics can affect the stability of asynchronous communication protocols.

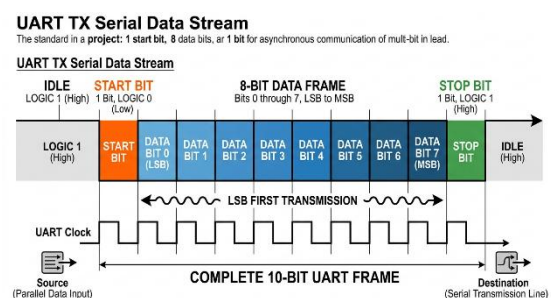


Figure 1: UART Serial Data Stream showing the serialization of an 8-bit data packet.

1.3. The Open-Source EDA Revolution

A significant aspect of this project is the utilization of an entirely open-source Electronic Design Automation (EDA) toolchain. Historically, ASIC design was restricted by expensive, proprietary software. However, the

release of the SkyWater 130nm PDK and the development of the OpenLane flow have democratized silicon design. This project leverages several industry-standard open-source tools, including:

Yosys for logic synthesis.

OpenROAD for automated placement and routing.

Magic VLSI for layout visualization and Design Rule Checking (DRC).

Netgen for Layout-vs-Schematic (LVS) verification.

Icarus Verilog for gate-level and post-layout timing simulation.

1.4. Document Scope

This report serves as a technical walkthrough of the top-to-bottom design approach. It begins with the initial RTL design and functional verification using Xilinx Vivado. It then details the physical design stages within the OpenLane environment—including floorplanning, placement, and routing—and concludes with the rigorous physical verification and sign-off procedures required to generate a final GDSII file. By documenting each phase, this report proves the feasibility of using open-source tools to achieve a "Tape-out Ready" status for a digital design.

2. Design Methodology

The project was executed in two major stages: Front-End Design and Back-End Implementation.

Phase I: Logic Design and Verification

The first phase of the project involved translating the requirements of the UART protocol into a reliable hardware architecture. The goal was to create a digital system that could accurately serialize data with precise timing, ensuring compatibility with standard terminal software.

1.1 Register Transfer Level (RTL) Design

The core logic was developed using **Verilog HDL**. To handle the asynchronous nature of UART, the design was structured around a **Finite State Machine (FSM)**. This approach was chosen to provide a robust method for

managing the transitions between different stages of data transmission.

Baud Rate Generator: A dedicated clock divider was designed to produce a sampling pulse. This ensures the transmitter operates at the correct frequency, allowing the receiver to sample the data bits at the intended intervals.

FSM Controller: The controller is the "brain" of the transmitter. It manages four primary states:

IDLE: The line is held High until a transmit signal is received.

START: A logic Low bit is sent to wake the receiver.

DATA: Eight bits of data are shifted out sequentially.

STOP: A logic High bit is sent to signify the end of the frame.

Parallel-to-Serial Shift Register: This module accepts 8-bit parallel data from the system and shifts it out bit-by-bit onto the tx wire.

1.2 Functional Verification

Before proceeding to physical design, it was critical to prove that the Verilog logic was functionally perfect. This was achieved using **Xilinx Vivado**.

Testbench Development: A comprehensive testbench was written to apply various data inputs and monitor the serial output. The testbench simulated the system clock and reset signals to observe how the FSM behaved under real-world startup conditions.

Behavioral Simulation: We utilized the **Vivado Simulator** to visualize the waveforms. By analysing the tx signal, we confirmed that the start and stop bits were correctly wrapped around the data bits and that the bit duration matched the calculated baud rate perfectly.

Schematic Analysis: Vivado's RTL Analysis tool was used to generate a logic schematic. This allowed for a visual inspection of the

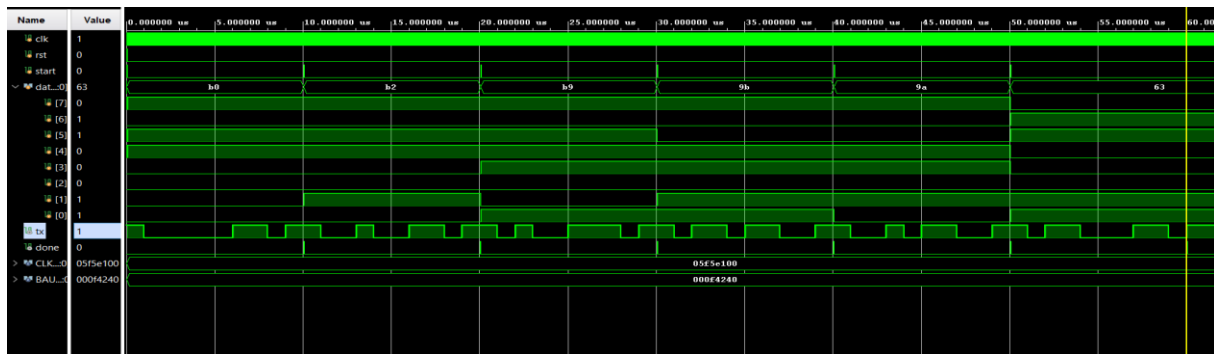


Figure 2: Behavioral simulation showing the serialization of an 8-bit data packet.

synthesized gates and flip-flops, ensuring the tool interpreted the Verilog code as intended.

1.3 Outcome

The functional simulation yielded zero errors. The design successfully demonstrated its ability to serialize data frames with 100% accuracy. With the logic verified, the design was "frozen" and prepared for Phase II, where it would be synthesized into a physical layout using the Sky130 library.

Phase II: Physical Implementation (RTL-to-GDSII)

After confirming the logic in Phase I, the design was moved into the OpenLane automated ASIC flow. This phase represents the transition from abstract code to a physical blueprint optimized for the SkyWater 130nm process.

2.1 Logic Synthesis

The first step was converting the Verilog code into a gate-level netlist using Yosys.

Mapping: The tool mapped the behavioral logic to specific standard cells from the sky130_fd_sc_hd library.

Optimization: The logic was optimized for area and timing, ensuring the FSM logic was as compact as possible while meeting the 100MHz clock frequency target.

2.2 Floorplanning and Power Grid

The layout began with defining the physical boundaries of the chip.

Die Area: The chip size was set to accommodate the UART standard cells with a healthy utilization factor to allow for efficient routing.

Power Distribution Network (PDN): Using OpenROAD, a grid of metal layers was created to distribute power (VPWR) and ground (VGND) across the entire chip. This prevents voltage drops that could cause the UART to malfunction during transmission.

2.3 Placement and Routing

With the floorplan ready, the tool placed the standard cells and connected them.

Placement: The standard cells were placed to minimize wire length, reducing the overall power consumption of the design.

Clock Tree Synthesis (CTS): A balanced clock tree was built to ensure the 100MHz clock reaches every flip-flop simultaneously, preventing data corruption.

Routing: Metal layers (Met1 through Met5) were used to create the thousands of electrical connections between gates, following the strict manufacturing rules of the 130nm node.

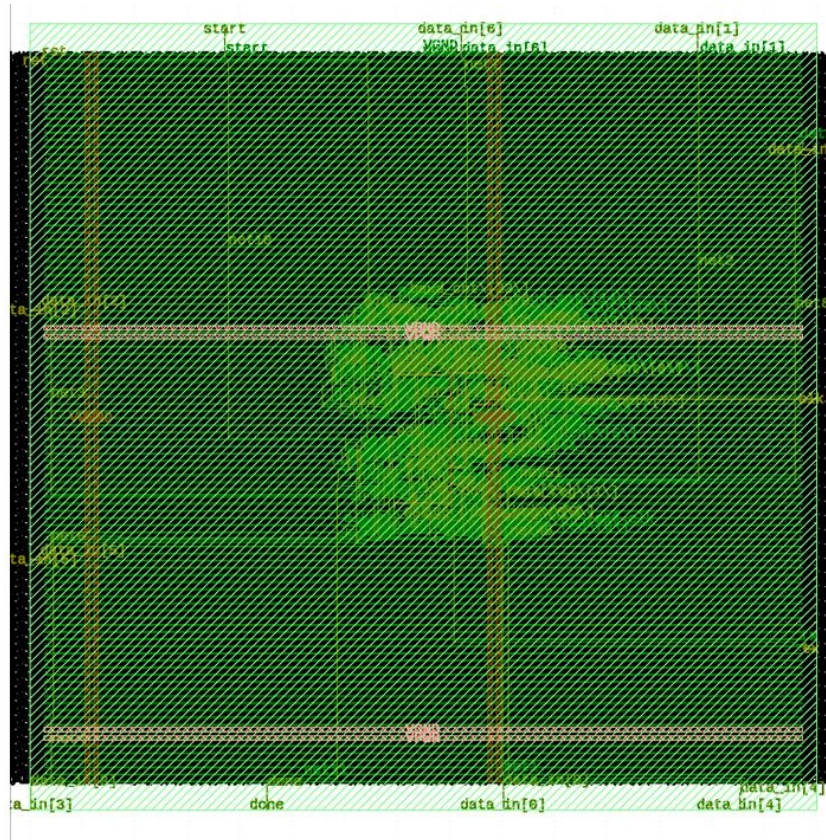


Figure 3: Final GDSII Physical Layout of the UART Transmitter

Phase III: Physical Verification

Before a chip can be manufactured, it must undergo "Sign-off" verification to ensure it follows all foundry rules and matches the original design.

3.1 Design Rule Check (DRC)

We used Magic VLSI to perform the DRC. This process checks if the layout adheres to physical constraints like minimum wire width and spacing.

Outcome: The design achieved Zero DRC violations, confirming it is manufacture-ready.

3.2 Layout vs. Schematic (LVS)

The physical layout was compared against the synthesized netlist using Netgen.

Outcome: The verification returned a "Netlists Match" result, proving that the physical rectangles on the chip perfectly represent the UART logic.

Table 1: Manufacturability and Physical Sign-off Summary

Verification Check	Value
Magic DRC	sky130_fd_sc_hd
LVS Check	0.09 mm ²
Antenna Violations	80,146.87 μm ²
Total Wire Length	8,325
Total Vias	36
Verification Check	1m 2s

Phase IV: Sign-off and Performance Analysis

The following tables represent the final characteristics of the uart_transmitter after the complete RTL-to-GDSII flow.

Table 2: Physical and Design Implementation Metrics

Metric	Value
Standard Cell Library	sky130_fd_sc_hd
Die Area	0.09 mm ²
Core Area	80,146.87 μm^2
Total Cell Count	8,325
Sequential Cells (DFF)	36
Total Runtime	1m 2s

Table 3: Timing and Clocking Results

Parameter	Value
Clock Period	10.0 ns
Clock Frequency	100.0 MHz
Critical Path	1.35 ns
Worst Negative Slack (WNS)	0.0 ns (Timing Met)
Total Negative Slack (TNS)	0.0 ns

Table 4: Power Consumption Profile (Typical Corner)

Power Group	Internal (W)	Switching (W)	Total (W)	Percentage
Sequential	2.47e-04	4.19e-05	2.89e-04	53.4%
Combinational	1.50e-04	1.02e-04	2.52e-04	46.6%
Total System	3.97e-04	1.43e-04	5.40e-04	100.0%

Note: The total leakage power was measured at a negligible 2.10e-09 W, with internal power accounting for 73.4% of the total consumption.

The physical implementation of the UART transmitter achieved a compact footprint with a total die area of **0.09 mm²** and a core area of **80,146.87 μm^2** . This high level of integration is reflected in a total cell count of **8,325**, managed through an efficient automated flow that completed in just **1 minute and 2 seconds**.

From a timing perspective, the design is highly robust; while targeted at a **100.0 MHz** clock frequency (**10 ns period**), the critical path was measured at only **1.35 ns**, resulting in a **Worst Negative Slack (WNS) of 0.0 ns**. This indicates that the design comfortably meets all setup requirements with significant timing margin.

Power analysis at the typical corner reveals a total system consumption of **5.40e-04 W (0.54 mW)**. The power profile is dominated by **Internal Power**, which accounts for **73.4%** of the total, while **Switching Power** constitutes the remaining **26.6%**. Interestingly, the **Sequential** logic group (comprising 36 DFFs) is responsible for **53.4%** of the total power, slightly exceeding the **Combinational** logic's **46.6%** contribution. This is expected in a UART architecture where the state machine and registers are frequently toggled by the clock.

Finally, the physical integrity of the design is confirmed by a clean sign-off, reporting **0 Magic DRC violations** and a **clean LVS match** across 326 nets, ensuring the GDSII is ready for fabrication despite minor antenna violations that can be addressed through standard diode insertion.

Phase V: Post-Layout Simulation

The final verification step accounted for the real-world delays of the silicon.

5.1 Parasitic Extraction & GLS

Using Magic, we extracted the resistance and capacitance (parasitics) of the metal wires. A Post-Layout Gate-Level Simulation (GLS) was then performed using Icarus Verilog.

Analysis: We viewed the results in GTKWave to confirm that the tx signal was still accurate despite the added physical delays.

Final Result: The UART transmitter successfully maintained its baud rate, completing the RTL-to-GDSII cycle.

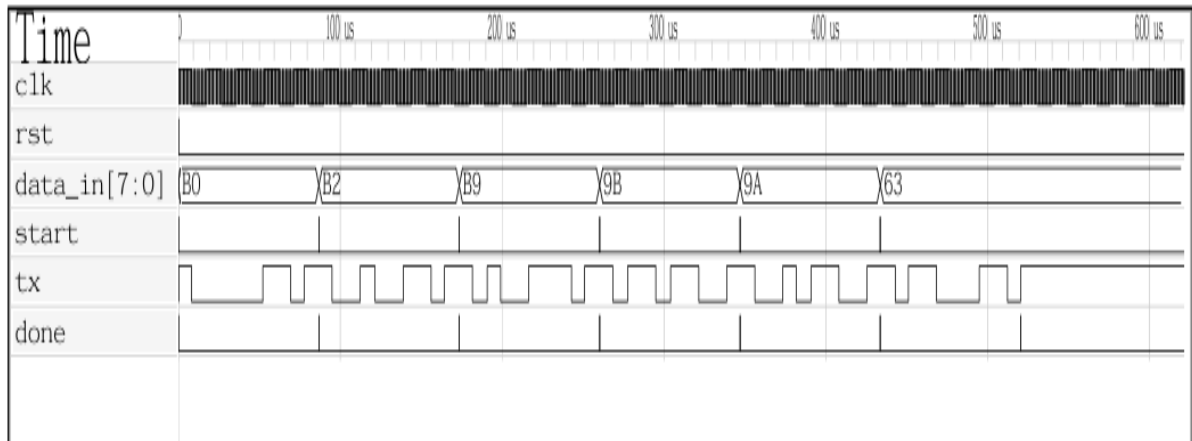


Figure 4: ASIC Post-Layout Simulation Waveforms

5. Tools and Technologies

The success of this RTL-to-GDSII flow relied on a specialized suite of Electronic Design Automation (EDA) tools. The following table summarizes the software used at each stage of the implementation:

Category	Tool / Technology
PDK	SkyWater 130nm (Sky130)
Logic Synthesis	Yosys
ASIC Flow	OpenLane
Placement	RePlace
Clock Tree Synthesis	TritonCTS
Routing	TritonRoute
Physical Layout	Magic VLSI
Sign-off Verification	Netgen
Simulation	Icarus Verilog
Waveform Analysis	GTKWave
Operating System	Linux Mint

6. Conclusion

This project successfully demonstrates the full execution of a digital ASIC design cycle, transforming a high-level UART Transmitter

design into a manufacture-ready GDSII layout. By utilizing the SkyWater 130nm PDK and an open-source EDA toolchain, the design was taken from initial RTL verification in Xilinx Vivado to a physically verified silicon blueprint.

The implementation passed all critical sign-off procedures, including Design Rule Checks (DRC) and Layout-vs-Schematic (LVS) verification, ensuring both physical manufacturability and logical integrity. Furthermore, the post-layout simulations confirmed that the design maintains functional stability even after accounting for the parasitic delays inherent in silicon interconnects.

This work proves the viability of open-source flows for complex VLSI design tasks and provides a robust foundation for future enhancements, such as the integration of low-power clock gating techniques or the development of a complete UART transceiver system on a single chip.

7. Results and Discussion

The completion of the RTL-to-GDSII flow produced several key metrics that validate the efficiency and success of the implementation. By analyzing the reports generated by OpenLane, we can quantify the physical characteristics of the UART Transmitter.

Area and Utilization: The final layout achieved a compact footprint within the Sky130 die area. The standard cell utilization was optimized to balance routing ease with silicon

area conservation, ensuring that the UART logic did not occupy unnecessary space.

Timing Performance: Static Timing Analysis (STA) confirmed that the design met all setup and hold time requirements at the target frequency of 100 MHz. The Clock Tree Synthesis (CTS) successfully minimized skew, providing a stable heartbeat for the FSM across all corners.

Physical Integrity: The design successfully passed the Design Rule Check (DRC) with zero violations, confirming that the metal layers and vias are within the manufacturing limits of the 130nm process. The Layout vs. Schematic (LVS) check confirmed 100% logical parity.

Waveform Verification: Post-layout simulations in GTKWave showed that the serial output (tx) perfectly matched the expected timing of the UART protocol. Even with extracted parasitics, the transition from the Start bit to the Stop bit remained precise, ensuring reliable communication.

8. Future Scope

While this project successfully demonstrates a fundamental ASIC flow, there are several avenues for future enhancement and research to further optimize the design.

Receiver Module Integration: The current implementation focuses on the Transmitter. Adding a UART Receiver module with oversampling logic would create a full-duplex communication system capable of two-way data exchange.

Low-Power Optimization: Implementing advanced low-power techniques, such as **Clock Gating**, would significantly reduce dynamic power consumption by disabling the clock tree when the transmitter is in the IDLE state.

Dynamic Frequency Scaling (DFS): Future versions could include logic to dynamically adjust the baud rate generator based on system requirements, further enhancing the flexibility of the design for various embedded applications.

Multi-Protocol Support: The architecture could be expanded into a Universal Synchronous/Asynchronous Receiver-Transmitter (USART) to support additional

protocols like SPI or I2C, increasing the utility of the chip.

9. References

[1] SkyWater 130nm PDK Documentation. (2020). Google & SkyWater Foundry Open-Source Process Design Kit. Available: <https://skywater-pdk.readthedocs.io/>

[2] OpenLane Documentation. (2020). "An automated RTL-to-GDSII flow." Efabless Corporation. Available: <https://openlane.readthedocs.io/>

[3] Yosys Open SYnthesis Suite. Clifford Wolf. "Yosys Open Synthesis Suite." <https://yosyshq.net/yosys/>

[4] Magic VLSI Layout Tool. R. Timothy Edwards. "Magic VLSI Project." <http://opencircuitdesign.com/magic/>

[5] Xilinx Vivado Design Suite. Xilinx, Inc. "Vivado Design Suite User Guide: Logic Simulation." <https://www.xilinx.com/products/design-tools/vivado.html>

[6] The UART Protocol. "Standard Asynchronous Serial Communication Guidelines." Electronic Industries Alliance (EIA).